

Access Specifier

(Public, Private and Protected)

Now before studying how to define class and its objects, let's first quickly learn what access are specifier. Access specifier in C++ class defines the access control rules. C++ has 3 new keywords introduced, namely,

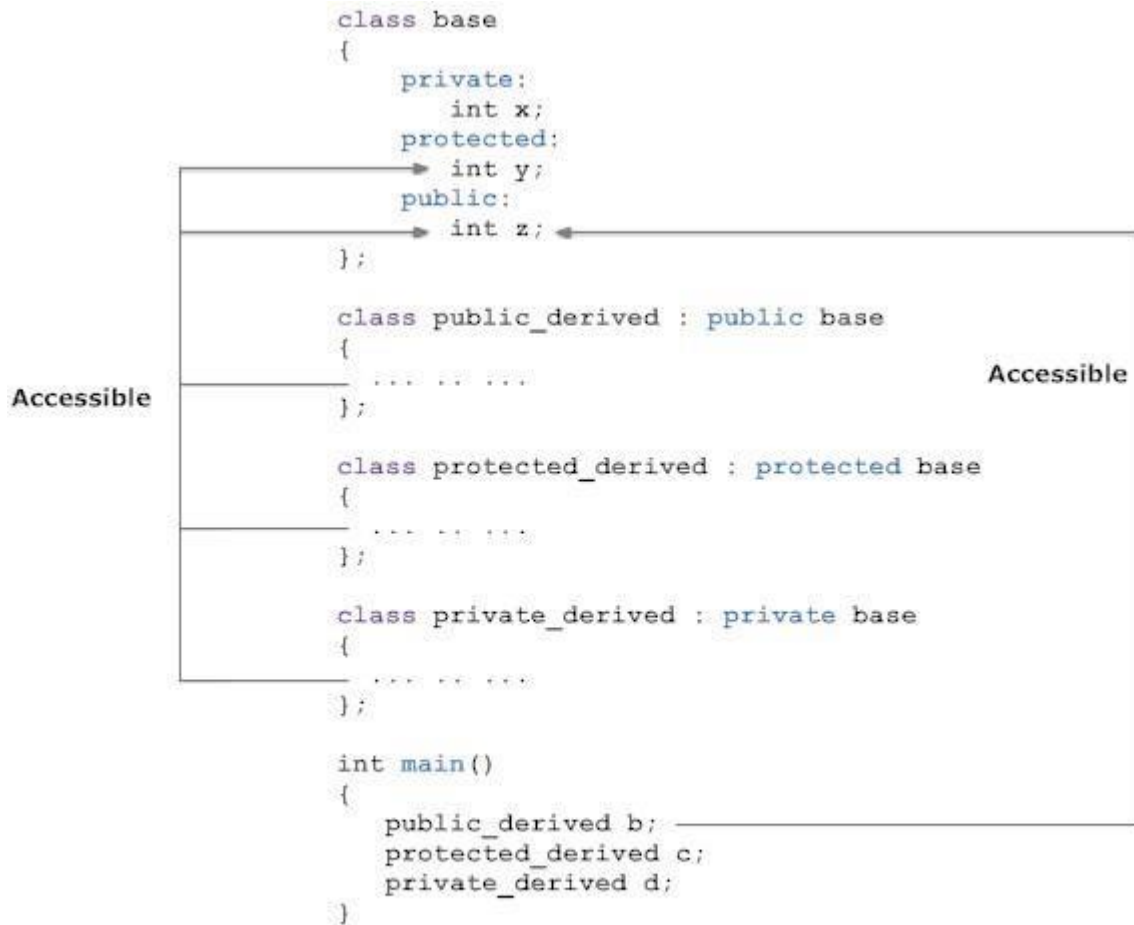
1. public
2. private
3. protected

- **Syntax:**

```
class base {  
    ....  
};  
  
class derived : access_specifier base {  
    ....  
};
```

- **Things to remember while Using Public, Protected and Private Inheritance**

1. Protected and public members (data and function) of a base class are accessible from a derived class (for all three: public, protected and private inheritance).
2. Objects of derived class with private and protected inheritance cannot access any data member of a base class.
3. Objects of derived class with public inheritance can access only public member of a base class.



This diagram shows all the access combination for public, private and protected inheritance. The arrow shows the position from where that particular member can be accessed.

These access specifiers are used to set boundaries for availability of members of class be it data members or member functions. Access specifiers in the program, are followed by a colon. You can use either one, two or all 3 specifiers in the same class to set different boundaries for different class members. They change the boundary for all the declarations that follow them.

❖ PUBLIC:

Public, means all the class members declared under public will be available to everyone. The data members and member functions declared public can be accessed by other classes too. Hence there are chances that they might change them. So the key members must not be declared public. A **public** member is accessible from

anywhere outside the class but within a program. You can set and get the value of public variables without any member function.

□ **Syntax:**

```
class Public Access
{ public: // public access specifier int x;
// Data Member Declaration void display(); //
Member Function declaration }
```

❖ **PRIVATE:**

Private keyword, means that no one can access the class members declared private outside that class. If someone tries to access the private member, they will get a compile time error. By default class variables and member functions are private.

□ **Syntax:**

```
class PrivateAccess
{ private: // private access specifier int x;
// Data Member Declaration void display(); //
Member Function declaration
}
```

A **private** member variable or function cannot be accessed, or even viewed from outside the class. Only the class and friend functions can access private members. By default all the members of a class would be private, for example in the following class **width** is a private member, which means until you label a member, and it will be assumed a private member:

```
class Box {  
    double width;  
    public:  
        double length;        void  
    setWidth ( double wid );  
    double getWidth ( void );  
};
```

❖ PROTECTED:

Protected, is the last access specifier, and it is similar to private, it makes class member inaccessible outside the class. But they can be accessed by any subclass of that class. (If class A is inherited by class B, then class B is subclass of class A. We will learn this later.)

A **protected** member variable or function is very similar to a private member but it provided one additional benefit that they can be accessed in child classes which are called derived classes.

□ Syntax:

```
class ProtectedAccess  
{ protected: // protected access specifier  
    int x;  
    // Data Member Declaration  
    void display(); // Member Function declaration  
}
```

❖ Summary of Public, Protected and Private Inheritance

- Accessibility in Public Inheritance

Accessibility	private	protected	public
Accessible from own class?	yes	yes	yes
Accessible from derived class?	no	yes	yes
Accessible outside derived class?	no	no	yes

- **Accessibility in Protected Inheritance**

Accessibility	private	protected	public
Accessible from own class?	yes	yes	yes
Accessible from derived class?	no	yes	yes
Accessible outside derived class?	no	no	no

- **Accessibility in Private Inheritance**

Accessibility	private	protected	public
Accessible from own class?	yes	yes	yes
Accessible from derived class?	no	yes	yes
Accessible outside derived class?	no	no	no